

Практическая работа 2. Списки, поиск, фильтрация и пагинация

Цель работы

Освоить управление состоянием через `provider`, разделение приложения на слои и построение списков с поиском, фильтрацией, сортировкой и постраничным выводом. Работа выполняется на данных в памяти — сервер подключается позже, в ПР4.

Что потребуется

- Приложение В «Управление состоянием через `provider`» целиком;
- Приложение Г «Адаптивная вёрстка и таблицы», разделы 3, 6–8.

Показательный проект

К этой работе относятся шаги `step-03–step-05` показательного проекта «Киберлига» (ссылка и общее описание — в программе практики, раздел 1.2): модели и данные в памяти, `provider` с поиском и фильтрами в параметрах адреса, адаптивная вёрстка со сменой таблицы на карточки.

```
git checkout step-04
git diff step-03 step-04
```

Предметная область другая, переносить код целиком незачем — смотрите именно на разделение по слоям и на то, как условия отбора превращаются в параметры адреса.

1. Постановка задачи

Начиная с этой работы и до конца практики разрабатывается одно приложение — клиент библиотечной системы. Предметная область описана в контракте учебного API (`api/КОНТРАКТ-API.md`), там же приведены поля всех сущностей.

В этой работе реализуется каталог: список книг и список авторов с полным набором операций просмотра. Данные хранятся в памяти, при перезагрузке страницы восстанавливаются из начального набора.

1.1. Соответствие конструкций

Java Spring

`InMemoryDAO` из материалов занятия

`Model` и передача атрибутов в шаблон

`Pageable`, `PageRequest.of(page, size)`

`<tr th:each="book : ${books}">`

Flutter Web

`InMemoryBookRepository`

`ChangeNotifier` И `context.watch`

СОБСТВЕННЫЙ КЛАСС `Page<T>`

`ListView.builder` ИЛИ `DataTable`

2. Ход выполнения

2.1. Модель

Модель — простой класс с полями, конструктором и методом `copyWith`. Метод `copyWith` нужен, чтобы изменять запись, не мутируя исходный объект: это упрощает отслеживание изменений.

```
// lib/models/book.dart

class Book {
  final int id;
  final String title;
  final String isbn;
  final int year;
  final int pages;
  final int publisherId;
  final List<int> authorIds;
  final List<int> genreIds;
  final int copiesTotal;
  final int copiesAvailable;
  final DateTime? deletedAt;

  const Book({
    required this.id,
    required this.title,
    required this.isbn,
    required this.year,
    required this.pages,
    required this.publisherId,
    required this.authorIds,
    required this.genreIds,
    required this.copiesTotal,
    required this.copiesAvailable,
    this.deletedAt,
  });

  bool get isDeleted => deletedAt != null;

  Book copyWith({
    String? title,
    String? isbn,
    int? year,
    int? pages,
    int? publisherId,
    List<int>? authorIds,
    List<int>? genreIds,
    int? copiesTotal,
    int? copiesAvailable,
    DateTime? deletedAt,
```

```

    bool clearDeletedAt = false,
  }) {
    return Book(
      id: id,
      title: title ?? this.title,
      isbn: isbn ?? this.isbn,
      year: year ?? this.year,
      pages: pages ?? this.pages,
      publisherId: publisherId ?? this.publisherId,
      authorIds: authorIds ?? this.authorIds,
      genreIds: genreIds ?? this.genreIds,
      copiesTotal: copiesTotal ?? this.copiesTotal,
      copiesAvailable: copiesAvailable ?? this.copiesAvailable,
      deletedAt: clearDeletedAt ? null : (deletedAt ?? this.deletedAt),
    );
  }
}

```

Обратите внимание на параметр `clearDeletedAt`. Без него нельзя отличить «не менять поле» от «установить `null`»: оба случая выглядят как переданный `null`. Это типичная ловушка `copyWith`, и на защите её стоит уметь объяснить.

2.2. Страница результатов

Постраничный вывод встречается во всех списках, поэтому его удобно описать один раз обобщённым типом. Дженерики разбирались в лекции 2.6.

```

// lib/models/page_result.dart

class PageResult<T> {
  final List<T> items;
  final int page;
  final int size;
  final int total;

  const PageResult({
    required this.items,
    required this.page,
    required this.size,
    required this.total,
  });

  int get totalPages => total == 0 ? 1 : (total / size).ceil();
  bool get hasPrevious => page > 1;
  bool get hasNext => page < totalPages;

  /// Пустая страница – начальное значение для состояния экрана.
  ///
  /// Конструктор намеренно не const: пустой константный список с параметром
  /// типа `T` в Dart запрещён, поэтому пишется обычный `[]`.
  PageResult.empty()
    : items = <T>[],
      page = 1,
      size = 10,
      total = 0;
}

```

Структура намеренно совпадает с оболочкой ответа учебного API. В ПР4 благодаря этому изменится только источник данных, а экраны останутся прежними.

2.3. Параметры запроса

Все условия отбора собираются в один неизменяемый объект. Это удобнее, чем передавать восемь отдельных аргументов, и позволяет положить их целиком в адрес.

```
// lib/models/book_query.dart
```

```
class BookQuery {
  final String search;
  final int? genreId;
  final int? publisherId;
  final int? yearFrom;
  final int? yearTo;
  final String sortField;
  final bool sortAscending;
  final int page;
  final int size;
  final bool includeDeleted;

  const BookQuery({
    this.search = '',
    this.genreId,
    this.publisherId,
    this.yearFrom,
    this.yearTo,
    this.sortField = 'title',
    this.sortAscending = true,
    this.page = 1,
    this.size = 10,
    this.includeDeleted = false,
  });

  BookQuery copyWith({
    String? search,
    Object? genreId = _unset,
    Object? publisherId = _unset,
    Object? yearFrom = _unset,
    Object? yearTo = _unset,
    String? sortField,
    bool? sortAscending,
    int? page,
    int? size,
    bool? includeDeleted,
  }) {
    return BookQuery(
      search: search ?? this.search,
      genreId: genreId == _unset ? this.genreId : genreId as int?,
      publisherId: publisherId == _unset ? this.publisherId : publisherId as int?,
      yearFrom: yearFrom == _unset ? this.yearFrom : yearFrom as int?,
      yearTo: yearTo == _unset ? this.yearTo : yearTo as int?,
    );
  }
}
```

```

    sortField: sortField ?? this.sortField,
    sortAscending: sortAscending ?? this.sortAscending,
    // Любое изменение условий отбора возвращает на первую страницу
    page: page ?? 1,
    size: size ?? this.size,
    includeDeleted: includeDeleted ?? this.includeDeleted,
  );
}

static const _unset = Object();
}

```

Приём с константой `_unset` решает ту же задачу, что и `clearDeletedAt` выше: он позволяет отличить «параметр не передан» от «передан `null`, сбросить фильтр».

Заметьте строку `page: page ?? 1`: при смене любого фильтра список обязан вернуться на первую страницу. Иначе пользователь, стоящий на седьмой странице, после ввода поискового запроса увидит пустой экран.

2.4. Репозиторий

Сначала описывается интерфейс — что приложение умеет делать с книгами, безотносительно того, где они лежат.

```

// lib/repositories/book_repository.dart

abstract interface class BookRepository {
  Future<PageResult<Book>> find(BookQuery query);
  Future<Book?> findById(int id);
  Future<Book> create(Book book);
  Future<Book> update(Book book);
  Future<void> softDelete(int id);
  Future<void> hardDelete(int id);
  Future<void> restore(int id);
  Future<int> deleteMany(List<int> ids);
}

```

Затем реализация на данных в памяти:

```

// lib/repositories/in_memory_book_repository.dart

class InMemoryBookRepository implements BookRepository {
  final List<Book> _books = [...seedBooks];
  int _nextId = seedBooks.length + 1;

  @override
  Future<PageResult<Book>> find(BookQuery q) async {
    // Небольшая задержка имитирует сеть: без неё индикатор загрузки
    // невозможно увидеть, а в ПР4 он понадобится по-настоящему.
    await Future.delayed(const Duration(milliseconds: 250));

    var rows = _books.where((b) => q.includeDeleted || !b.isDeleted).toList();

    if (q.search.trim().isEmpty) {
      final needle = q.search.trim().toLowerCase();
      rows = rows
    }
  }
}

```

```

        .where((b) =>
            b.title.toLowerCase().contains(needle) ||
            b.isbn.toLowerCase().contains(needle))
        .toList();
    }

    if (q.genreId != null) {
        rows = rows.where((b) => b.genreIds.contains(q.genreId)).toList();
    }
    if (q.publisherId != null) {
        rows = rows.where((b) => b.publisherId == q.publisherId).toList();
    }
    if (q.yearFrom != null) rows = rows.where((b) => b.year >= q.yearFrom!).toList();
    if (q.yearTo != null) rows = rows.where((b) => b.year <= q.yearTo!).toList();

    rows.sort((a, b) {
        final result = switch (q.sortField) {
            'year' => a.year.compareTo(b.year),
            'pages' => a.pages.compareTo(b.pages),
            _ => a.title.toLowerCase().compareTo(b.title.toLowerCase()),
        };
        return q.sortAscending ? result : -result;
    });

    final total = rows.length;
    final from = (q.page - 1) * q.size;
    // Осторожно: num.clamp возвращает num, а не int, и в sublist его
    // передать нельзя. Границу считаем явно.
    final to = (from + q.size) > total ? total : (from + q.size);
    final items = from >= total ? <Book>[] : rows.sublist(from, to);

    return PageResult(items: items, page: q.page, size: q.size, total: total);
}

@override
Future<void> softDelete(int id) async {
    final i = _books.indexWhere((b) => b.id == id);
    if (i == -1) throw StateError('Книга $id не найдена');
    _books[i] = _books[i].copyWith(deletedAt: DateTime.now());
}

@override
Future<void> hardDelete(int id) async {
    _books.removeWhere((b) => b.id == id);
}

@override
Future<void> restore(int id) async {
    final i = _books.indexWhere((b) => b.id == id);
    if (i == -1) throw StateError('Книга $id не найдена');
    _books[i] = _books[i].copyWith(clearDeletedAt: true);
}

@override
Future<int> deleteMany(List<int> ids) async {
    var count = 0;
    for (final id in ids) {

```

```

        final i = _books.indexWhere((b) => b.id == id && !b[i].isDeleted);
        if (i != -1) {
            _books[i] = _books[i].copyWith(deletedAt: DateTime.now());
            count++;
        }
    }
    return count;
}

// create, update, findById реализуются самостоятельно
}

```

Внимание. В методе `deleteMany` намеренно оставлена ошибка. Найдите и исправьте её — это часть задания на оценку «4». Приложение с ней не скомпилируется; сообщение анализатора подскажет направление.

2.5. Состояние экрана

```

// lib/state/book_list_notifier.dart

enum LoadStatus { idle, loading, success, error }

class BookListNotifier extends ChangeNotifier {
    final BookRepository _repository;

    BookListNotifier(this._repository);

    BookQuery _query = const BookQuery();
    PageResult<Book> _result = PageResult.empty();
    LoadStatus _status = LoadStatus.idle;
    String? _error;
    final Set<int> _selected = {};

    BookQuery get query => _query;
    PageResult<Book> get result => _result;
    LoadStatus get status => _status;
    String? get error => _error;
    Set<int> get selected => Set.unmodifiable(_selected);
    bool get hasSelection => _selected.isNotEmpty;

    Future<void> load() async {
        _status = LoadStatus.loading;
        _error = null;
        notifyListeners();

        try {
            _result = await _repository.find(_query);
            _status = LoadStatus.success;
        } catch (e) {
            _error = 'Не удалось загрузить список: $e';
            _status = LoadStatus.error;
        }
        notifyListeners();
    }

    Future<void> applyQuery(BookQuery next) async {
        _query = next;
    }
}

```

```

    _selected.clear(); // выделение теряет смысл при смене условий отбора
    await load();
  }

  void toggleSelection(int id) {
    _selected.contains(id) ? _selected.remove(id) : _selected.add(id);
    notifyListeners();
  }

  Future<void> deleteSelected() async {
    await _repository.deleteMany(_selected.toList());
    _selected.clear();
    await load();
  }
}

```

2.6. Подключение провайдеров

```

// lib/main.dart
void main() {
  usePathUrlStrategy();
  runApp(
    MultiProvider(
      providers: [
        Provider<BookRepository>(create: (_) => InMemoryBookRepository()),
        ChangeNotifierProvider(
          create: (context) =>
            BookListNotifier(context.read<BookRepository>())..load(),
        ),
      ],
      child: const LibraryApp(),
    ),
  );
}

```

2.7. Переиспользуемая таблица

Ключевое требование работы. Списков в приложении будет много — книги, авторы, жанры, издательства, читатели, выдачи. Писать для каждого отдельный экран со своей таблицей, своим поиском и своей пагинацией означает шесть почти одинаковых файлов по триста строк. К итоговому проекту с восемью сущностями такой подход становится неподъёмным.

Правильное решение — один обобщённый виджет, настраиваемый описанием колонок:

```

// lib/widgets/entity_table.dart

class TableColumnSpec<T> {
  final String label;
  final String? sortField;
  final bool numeric;
  final Widget Function(T item) build;

  const TableColumnSpec({
    required this.label,

```

```

        required this.build,
        this.sortField,
        this.numeric = false,
    });
}

class EntityTable<T> extends StatelessWidget {
  final List<TableColumnSpec<T>> columns;
  final List<T> items;
  final int Function(T item) idOf;
  final Set<int> selected;
  final ValueChanged<int>? onToggleSelect;
  final String? sortField;
  final bool sortAscending;
  final void Function(String field)? onSort;
  final List<Widget> Function(T item)? actions;

  const EntityTable({
    super.key,
    required this.columns,
    required this.items,
    required this.idOf,
    this.selected = const {},
    this.onToggleSelect,
    this.sortField,
    this.sortAscending = true,
    this.onSort,
    this.actions,
  });

  @override
  Widget build(BuildContext context) {
    // Реализация: DataTable с двумерной прокруткой,
    // см. приложение Г, раздел 8.1
    ...
  }
}

```

Применение для книг и для авторов при этом различается только описанием колонок:

```

EntityTable<Book>(
  items: notifier.result.items,
  idOf: (b) => b.id,
  sortField: notifier.query.sortField,
  sortAscending: notifier.query.sortAscending,
  onSort: (field) => notifier.applyQuery(
    notifier.query.copyWith(
      sortField: field,
      sortAscending: field == notifier.query.sortField
        ? !notifier.query.sortAscending
        : true,
    ),
  ),
  columns: [
    TableColumnSpec(label: 'Название', sortField: 'title', build: (b) =>
      Text(b.title)),
  ],
)

```

```

    TableColumnSpec(label: 'Год', sortField: 'year', numeric: true, build: (b) =>
Text('${b.year}')),
    TableColumnSpec(label: 'Страниц', sortField: 'pages', numeric: true, build: (b) =>
Text('${b.pages}')),
  ],
  actions: (b) => [
    IconButton(icon: const Icon(Icons.edit), onPressed: () =>
context.go('/books/${b.id}/edit')),
    IconButton(icon: const Icon(Icons.delete), onPressed: () =>
_confirmDelete(context, b)),
  ],
)

```

3. Задание

На оценку «3»

1. Реализованы две сущности: Book и Author. Для каждой есть модель с `copyWith` и репозиторий в памяти с начальным набором не менее чем из 20 записей для книг и 8 для авторов.
2. Приложение разделено на слои `models`, `repositories`, `state`, `screens`, `widgets`. Виджеты не обращаются к репозиторию напрямую.
3. Состояние управляется через `provider`; `setState` допускается только для локального состояния виджета вроде раскрытия панели.
4. Реализованы экраны списка и карточки для обеих сущностей, переходы через `go_router`.
5. Обрабатываются все четыре состояния экрана: загрузка, успешная загрузка, пустой результат, ошибка. Пустой список и ошибка выглядят по-разному.
6. `flutter analyze` не выдаёт замечаний.

На оценку «4»

Дополнительно:

7. Исправлена ошибка, намеренно оставленная в методе `deleteMany` (раздел 2.4). В отчёте описано, в чём она заключалась.
8. Реализован поиск по полю, отличному от идентификатора: для книг по названию и ISBN, для авторов по фамилии и стране.
9. Реализована фильтрация не менее чем по трём критериям одновременно: жанр, издательство, диапазон года издания. Фильтры комбинируются между собой и с поиском.
10. Реализовано множественное удаление: флажки в строках, счётчик выбранного, кнопка удаления выбранных с запросом подтверждения.
11. Реализованы оба вида удаления. Логическое проставляет `deletedAt` и убирает запись из выборки; физическое стирает её насовсем.

Предусмотрен переключатель показа удалённых записей и операция восстановления.

12. Раскладка адаптивна: при ширине окна менее 600 пикселей вместо таблицы показывается список карточек, при большей — таблица.

На оценку «5»

Дополнительно:

13. Реализован постраничный вывод с размером страницы не менее 10 записей. Присутствуют переходы на первую, предыдущую, следующую и последнюю страницы, показан номер текущей страницы и общее число записей. Размер страницы переключается пользователем: 10, 25, 50.
14. Реализована сортировка по щелчку на заголовке колонки, не менее чем по трём полям, с переключением направления и отображением стрелки в активной колонке.
15. Таблица вынесена в обобщённый виджет `EntityTable<T>` и используется **и для книг, и для авторов** без копирования кода.
16. Все условия отбора отражаются в адресе:
`/books?search=война&genreId=2&sort=year,desc&page=3`. Открытие такого адреса в новой вкладке восстанавливает то же состояние списка. Кнопка «назад» браузера возвращает к предыдущему набору условий.
17. Поле поиска не запускает выборку на каждое нажатие клавиши: реализована задержка не менее 300 миллисекунд после окончания ввода.

Пункт 16 — главное отличие web-приложения от мобильного. Ссылку на отфильтрованный список должно быть можно отправить коллеге.

4. Что сдать

1. Ссылка на репозиторий.
2. Снимки экранов: список в виде таблицы, список в виде карточек на узком окне, применённые фильтры, выделение нескольких записей, пустой результат, состояние ошибки.
3. Снимок адресной строки с параметрами отбора.
4. Описание найденной в `deleteMany` ошибки.
5. Раздел отчёта со схемой слоёв приложения и обоснованием, почему таблица вынесена в обобщённый виджет.

5. Типичные ошибки

Симптом

Список не обновляется после удаления

Причина

не вызван `notifyListeners()` либо не перезапрошены данные

Симптом

После ввода в поиск открывается пустая страница

Индикатор загрузки не исчезает

При пустом результате показан индикатор загрузки

Could not find the correct Provider

Таблица уезжает за край окна

Выделение сохраняется после смены фильтра и удаляются не те записи

Приложение подтормаживает при вводе в поиск

Причина

не сброшен номер страницы при смене условий отбора

`notifyListeners()` вызван только в начале загрузки

состояние «успех с пустым списком» не отделено от «загружается»

провайдер объявлен ниже виджета, который его запрашивает

не задана двумерная прокрутка, см. приложение Г, раздел 8.1

не очищен набор выбранных при изменении условий отбора

выборка запускается на каждый символ, не реализована задержка